

Documented by : Rajshree Thapa

Solution

The solution is designed to build an end-to-end **data pipeline** using AWS and Big Data tools.

Step 1: Data Ingestion

- Upload raw CSV datasets (Patients, Subscriber, Claims, Group_subgroup) to **AWS S3** under:

s3://<bucket-name>/input-data/

Step 2: Data Processing & Cleaning (PySpark)

Using PySpark (Databricks or EMR):

- Read data from S3
 - Perform cleaning:
 - Replace NULL values with "NA"
 - Remove duplicate records
 - Standardize column formats
 - Validate data quality
-

Step 3: Data Transformation

- Convert cleaned data into structured format
 - Apply necessary transformations:
 - Type casting (age, amount, etc.)
 - Filtering invalid records
 - Joining datasets where needed
-

Step 4: Load Data into Redshift

- Create staging tables in Redshift
- Load cleaned datasets into respective tables:
 - patients

- subscriber
 - claims
 - group_subgroup
-

Step 5: Analytics Layer (Output Tables)

- Create a new schema:

Project-Output

- Create output tables for each use case:
 - disease_max_claims
 - subscribers_under_30
 - most_profitable_group
 - claims_by_city
 - rejected_claims
 - etc.
-

Step 6: Query & Reporting

- Run SQL queries in Redshift
 - Generate insights for business
 - Store results in output tables
-

Step 7: Deployment

- Code pushed to GitHub
 - Pipelines executed using EMR/Databricks
 - Final outputs verified
-

2. Use Cases

The system supports the following business use cases:

1. Identify disease with maximum number of claims
2. Find subscribers below age 30

3. Identify group with maximum subgroups
 4. Find hospital serving most patients
 5. Identify most subscribed subgroup
 6. Count rejected claims
 7. Identify city with highest claims
 8. Compare government vs private subscriptions
 9. Calculate average monthly premium
 10. Identify most profitable group
 11. Find patients below 18 with cancer
 12. Identify high-cost cashless patients
 13. Female patients over 40 with knee surgery
-

3. Database Design (Redshift)

a. Tables Metadata with PK/FK

Patients Table

- patient_id (PK)
 - name
 - age
 - gender
 - disease
 - hospital
-

Subscriber Table

- subscriber_id (PK)
 - patient_id (FK)
 - age
 - group_id
 - subgroup_id
 - premium_amount
-

Claims Table

- claim_id (PK)
 - patient_id (FK)
 - disease
 - claim_amount
 - claim_status
 - city
-

Group_Subgroup Table

- group_id (PK)
 - subgroup_id
 - group_type (Govt/Private)
-
-

Output Tables (Project-Output Schema)

Example:

disease_max_claims

- disease
- total_claims

claims_by_city

- city
- total_claims

most_profitable_group

- group_id
 - total_revenue
-

b. ER Diagram (Optional)

4. Technologies and Platforms

Layer	Technology
Storage	AWS S3
Processing	PySpark
Compute	AWS EMR / Databricks
Data Warehouse	AWS Redshift
Version Control	GitHub
Project Tracking	Jira